UART y RS-232 con Raspberry Pi Pico

01 Ana Nickole Cisneros Herrera · 203477

02 Leslie Yoselin Sánchez Flores · 201831

03 RP2040 y RP2350 · Acceso por registros y Pico SDK

Una computadora controla un sistema embebido

01 La PC envía texto y la Pico ejecuta comandos.

02 TX y RX transportan bytes en sentidos independientes.

03 El resultado aparece en la terminal y en un LED externo.

UART organiza las tramas

01 Universal Asynchronous Receiver/Transmitter.

02 Convierte bytes paralelos en una secuencia de bits y viceversa.

03 Ambos extremos acuerdan baud, bits, paridad y stop.

RS-232 adapta la interfaz eléctrica

01 GP0 y GP1 trabajan con lógica de 3.3 V.

02 MAX3232 convierte niveles e invierte la polaridad.

03 El USB–RS-232 de la PC termina el enlace serial.

De los terminales a la UART integrada

01 Los terminales y módems necesitaban intercambio serial fiable.

02 Circuitos externos realizaban la conversión paralelo–serie.

03 Los microcontroladores integran UART, FIFO y control por registros.

Evolución de la interfaz

01 La estandarización facilitó conectar equipos de distintos fabricantes.

02 Las bombas de carga permiten RS-232 desde una alimentación baja.

03 FIFO, interrupciones y DMA facilitan atender flujos de datos.

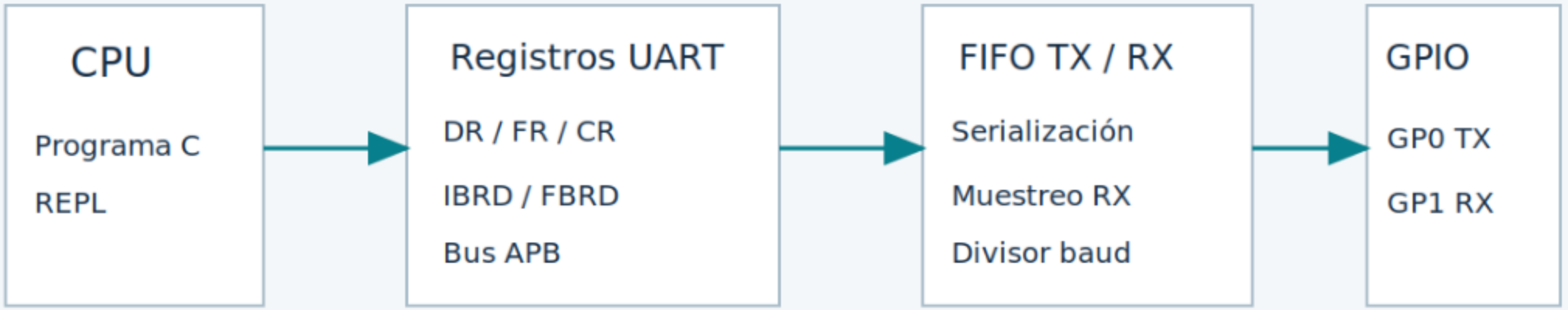
Aplicaciones que siguen utilizando UART

01 Consolas de diagnóstico en equipos embebidos.

02 Módulos GNSS y módems con comandos de texto.

03 Instrumentos, balanzas y controladores con RS-232.

Arquitectura de UART0



clk_peri alimenta el generador de baud. FIFO: 32 entradas por dirección.

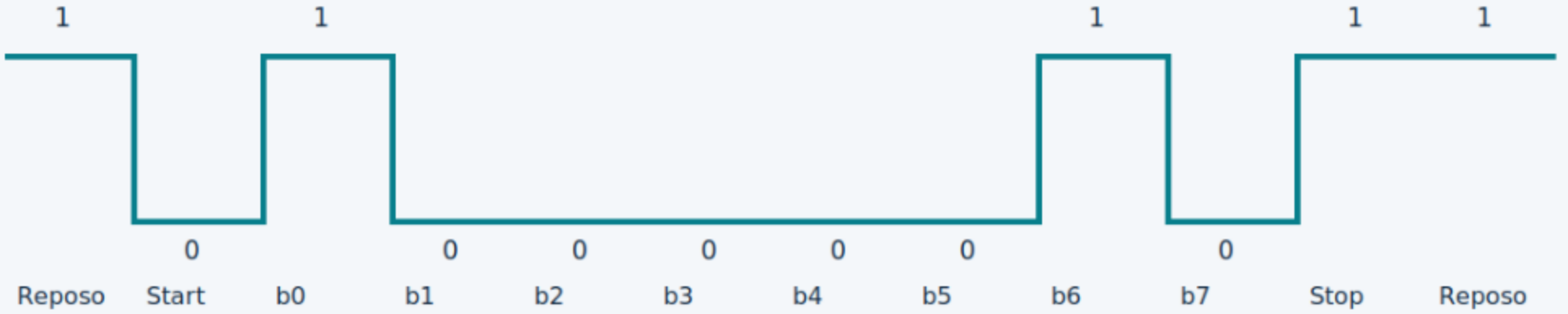
CPU y bus APB acceden a registros de control y datos.

FIFO TX/RX separan el acceso de CPU del desplazamiento de bits.

El divisor usa clk_peri; el multiplexor conecta UART con GP0/GP1.

Una trama 8N1 transporta un byte

Carácter A = 0x41, transmitido con 8N1



Se muestra el lado lógico de 3.3 V. En RS-232 la polaridad se invierte.

Inicio: 0 · Datos: 8 bits, menos significativo primero.

Paridad: ninguna · Stop: 1 · Reposo lógico: alto.

9600 bits/s: 104.17 µs/bit y unos 960 bytes/s útiles.

Opciones de configuración

01 5–8 bits; paridad ninguna, par o impar; 1–2 stop.

02 FIFO, RTS/CTS, interrupciones y DMA según la aplicación.

03 Nuestro enlace usa polling, 9600 8N1 y ningún control de flujo.

Dirección base y offset

01 Cada registro ocupa una dirección del mapa de memoria.

02 Dirección del registro = UART0_BASE + offset.

03 CR en RP2350: 0x40070000 + 0x030 = 0x40070030.

Registros de datos y estado

Registro	Offset	Uso
DR	0x000	Escribir TX / leer RX
FR	0x018	RXFE[4], TXFF[5], BUSY[3]
IBRD / FBRD	0x024 / 0x028	Divisor entero y fracción
LCR_H	0x02C	Formato y FIFO
CR	0x030	Habilitar UART, TX y RX

Campos y valores de la aplicación

Registro	Valor	Significado
LCR_H	0x70	WLEN=3; FEN=1; PEN=0; STP2=0
CR	0x301	RXE=1; TXE=1; UARTEN=1
IMSC / DMACR	0 / 0	Sin IRQ UART ni DMA
ICR	0x7FF	Limpia interrupciones pendientes
RSR / ECR	Leer / escribir	Estado / limpieza de errores

Un puntero accede al registro

```
01  volatile uint32_t *cr =
02      (volatile uint32_t *)(UART0_BASE + 0x030u);
03  *cr = (1u << 9) | (1u << 8) | 1u;
```

Inicialización por registros

01 Reset UART0 y espera de RESET_DONE.

02 IBRD y FBRD calculan baud; LCR_H aplica formato.

03 CR habilita recepción y transmisión después de configurar.

El SDK configura el mismo periférico

```
01  uart_init(uart0, 9600);

02  uart_set_format(uart0, 8, 1, UART_PARITY_NONE);

03  uart_set_hw_flow(uart0, false, false);
```

Transmitir y recibir en ambos modos

Acción	Registros	Pico SDK
Transmitir	Esperar TXFF=0, escribir DR	uart_putc_raw
Recibir	Esperar RXFE=0, leer DR	uart_getc
Terminar TX	Esperar BUSY=0	uart_tx_wait_blocking
Configurar pines	Infraestructura compartida	gpio_set_function

Qué abstrae el SDK

01 Nombres de funciones y macros sustituyen direcciones manuales.

02 El código resulta más portable entre familias.

03 Los registros siguen siendo la base del funcionamiento.

RP2040 y RP2350 cambian de mapa

Elemento	RP2040	RP2350
UART0_BASE	0x40034000	0x40070000
UART1_BASE	0x40038000	0x40078000
RESETS_BASE	0x4000C000	0x40020000
RESET UART0	bit22	bit26
CPU del build	Cortex-M0+	Cortex-M33

Lo que se conserva

01 Los offsets DR, FR, IBRD, FBRD, LCR_H y CR usados.

02 UART0 en GP0/GP1 y formato de comunicación 8N1.

03 Una fuente común con compilaciones independientes.

Un divisor calculado desde el reloj

01 D = clk_peri / (16 × baud).

02 IBRD contiene la parte entera; FBRD fracciones de 1/64.

03 Ejemplo a 48 MHz y 9600 baud: IBRD=312, FBRD=32.

El monitor del curso sigue reconocible

01 Tabla command_t y handlers conservados.

02 split_line crea argc y argv; execute_command elige la función.

03 Nuevos comandos: uart y rs232.

Comandos y validación

01 uart info · uart regs · uart sdk

02 rs232 hola desde la Pico

03 Línea máxima: 63 caracteres ASCII; máximo: 16 argumentos.

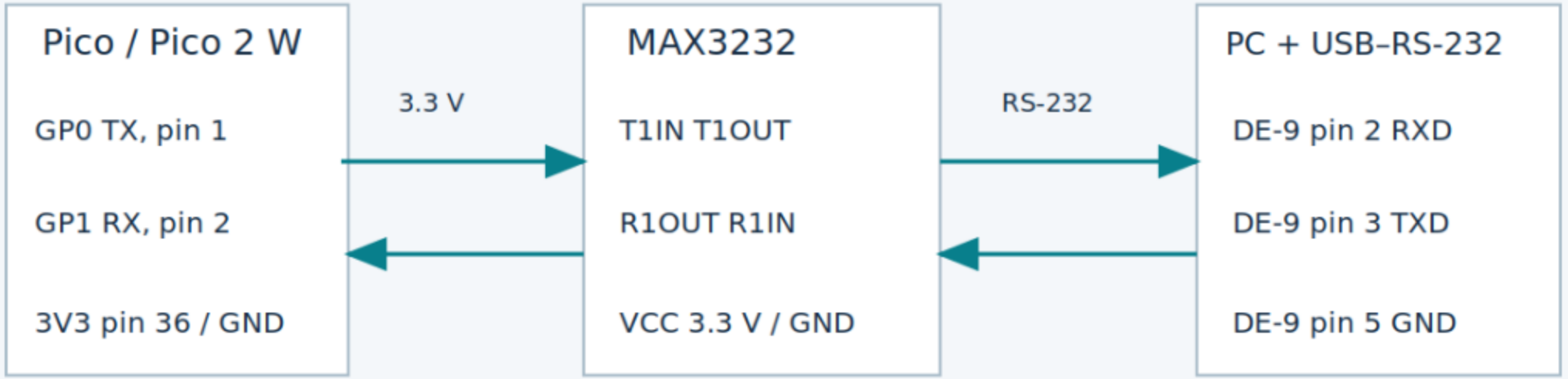
El recorrido completo de un comando

01 La terminal envía la orden por el adaptador RS-232.

02 UART recibe bytes y el REPL llama al handler de C.

03 La respuesta [RS232] confirma que el programa procesó el texto.

Conexiones de la demostración



GND común. Alimentación desde Pico. Pinout del adaptador: DTE estándar.

Verificar continuidad si el módulo ya trae DE-9. El USB de Pico no transporta el REPL.

MAX3232 alimentado desde 3V3 OUT.

TX de Pico a T1IN; R1OUT a RX de Pico.

Pinout DE-9 mostrado para un adaptador DTE estándar.

Prueba en Pico 2 W

01 uart info → rs232 prueba regs

02 uart sdk → rs232 prueba sdk

03 on → off → uart regs

Prueba en Pico RP2040

01 Repetir consulta y envío en regs y sdk.

02 Comprobar base 0x40034000.

03 Capturar terminal y montaje real de ambas plataformas.

Conclusiones y límites

01 UART configura y transporta los bytes de la aplicación.

02 MAX3232 permite comunicar lógica de 3.3 V con RS-232.

03 Registros y SDK ofrecen dos formas de controlar el mismo bloque.

Referencias técnicas

01 Raspberry Pi: datasheets RP2040 y RP2350.

02 Raspberry Pi: Pico SDK 2.2.0 y Hardware APIs.

03 Texas Instruments: MAX3232 SLLS410O.

04 Analog Devices: Fundamentals of RS-232 Serial Communications.